

λ -calcul typé homoïconique

Rawšanī

1. Définitions

On distinguera dans la suite un *contexte* et une *structure*, la seconde étant la réification de la première.

Contexte. On définit inductivement un contexte Γ de la façon suivante :

$$\Gamma ::= \emptyset \mid \Gamma, x : \tau$$

Un contexte n'est pas un type, c'est une donnée de la méta-théorie du langage qui n'est donc pas manipulable par le langage lui-même. C'est dans cet objectif dernièrement évoqué que l'on introduit ci-après la notion de *structure*.

Structure. On définit une structure comme la réification d'un contexte Γ par un type **Struct** Γ , lui-même un type produit :

$$\text{Struct } \emptyset = \mathbf{1} \qquad \text{Struct } (\Gamma, x : \tau) = \tau \times \text{Struct } \Gamma$$

On munit ce type des constructeurs de terme correspondants **snil** et **scons**, définis comme suit :

$$\text{snil} : \text{Code } \Gamma \text{ (Struct } \emptyset)$$

$$\text{scons} : \forall \tau. \text{Sym } x \rightarrow \tau \rightarrow \text{Code } \Gamma \text{ (Struct } \Delta) \rightarrow \text{Code } \Gamma \text{ (Struct } (\Delta, x : \tau))$$

On a donc, pour $\Gamma = \emptyset$, $a : \tau$, $b : \sigma$:

$$\text{scons } b \ \sigma \ (\text{scons } a \ \tau \ \text{snil}) \ : \ \text{Code } \Gamma \text{ (Struct } (\emptyset, a:\tau, b:\sigma))$$

Pour $\Gamma = \emptyset$, $a : \tau$, $b : \sigma$, l'évaluation de ce terme produit :

$$\{a := v_a, b := v_b\} : \text{Struct } \Gamma = \llbracket \Gamma \rrbracket = \sigma \times (\tau \times \mathbf{1})$$

avec $v_a : \tau$ et $v_b : \sigma$.

Code. On définit **Code** $\Gamma \ \tau$ comme le type des termes de calcul, avec Γ le contexte où sont recensés les symboles libres de l'expression et τ le type produit par l'expression, dès que son contexte est fermé, *i.e.* $\Gamma = \emptyset$. C'est donc le type correspondant à un « programme », réifié comme une donnée du langage. Cette réification typée constitue le cœur de la solution au problème du typage d'une donnée homoïconique, et rend possible le typage statique du code cité.

Formellement, $\text{Code } \Gamma \tau$ est un type somme, dont chaque sommande correspond à l'un des constructeurs suivants :

$$\begin{aligned} \text{App} &: \text{Code } \Gamma (\sigma \rightarrow \tau) \rightarrow \text{Code } \Gamma \sigma \rightarrow \text{Code } \Gamma \tau \\ \text{Abs} &: \text{Sym } x \rightarrow \text{Code } (\Gamma, x : \sigma) \tau \rightarrow \text{Code } \Gamma (\sigma \rightarrow \tau) \\ \text{Var} &: \text{Sym } x \rightarrow x : \tau \in \Gamma \rightarrow \text{Code } \Gamma \tau \\ \text{Cst} &: \tau \rightarrow \text{Code } \Gamma \tau \\ \text{EvalIn} &: \text{Code } \Gamma (\text{Code } \Delta \tau) \rightarrow \text{Code } \Gamma (\text{Struct } \Delta) \rightarrow \text{Code } \Gamma \tau \\ \text{CurrentCtx} &: \text{Code } \Gamma (\text{Struct } \Gamma) \end{aligned}$$

soit, en somme,

$$\text{Code } \Gamma \tau = \text{App}(\dots) + \text{Abs}(\dots) + \text{Var}(\dots) + \text{Cst}(\dots) + \dots$$

où chaque sommande porte les arguments définis juste avant.

L'adéquation du système repose sur le fait que toute valeur $e : \text{Code } \Gamma \tau$, quelle que soit la façon dont elle a été construite, dénote nécessairement un terme bien typé de type τ dès lors qu'on lui fournit, via **eval-in**, une structure satisfaisant son contexte, soit :

$$e : \text{Code } \Gamma \tau \text{ et } s : \text{Struct } \Gamma \implies \text{eval-in } e \ s : \tau$$

2. Type System

Le système de types est donné par les règles suivantes.

Les règles **HERE** et **THERE** définissent l'appartenance d'une variable à un contexte :

$$\frac{}{x : \tau \in (\Gamma, x : \tau)} \quad (\text{HERE})$$

$$\frac{x : \tau \in \Gamma}{x : \tau \in (\Gamma, y : \sigma)} \quad (\text{THERE})$$

On en déduit directement la règle de typage des variables :

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \quad (\text{VAR})$$

Les abstractions étendent le contexte avec leur paramètre :

$$\frac{\Gamma, x : \tau \vdash e : \sigma}{\Gamma \vdash \lambda x : \tau. e : \tau \rightarrow \sigma} \quad (\text{ABS})$$

et les applications imposent la compatibilité entre le type de la fonction et celui de son argument :

$$\frac{\Gamma \vdash f : \tau \rightarrow \sigma \quad \Gamma \vdash e : \tau}{\Gamma \vdash f e : \sigma} \quad (\text{APP})$$

Les constantes de base sont typées par :

$$\frac{}{\Gamma \vdash \star : \mathbf{1}} \quad (\text{UNIT})$$

$$\frac{i \in \mathbb{Z}}{\Gamma \vdash i : \text{Int}} \quad (\text{INT})$$

$$\frac{c \in \{\top, \perp\}}{\Gamma \vdash c : \text{Bool}} \quad (\text{BOOL})$$

Les produits se construisent et se décomposent selon :

$$\frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \sigma}{\Gamma \vdash \langle e_1, e_2 \rangle : \tau \times \sigma} \quad (\text{PAIR})$$

$$\frac{\Gamma \vdash e : \tau \times \sigma}{\Gamma \vdash \pi_1 e : \tau} \quad (\text{FST})$$

$$\frac{\Gamma \vdash e : \tau \times \sigma}{\Gamma \vdash \pi_2 e : \sigma} \quad (\text{SND})$$

Les sommes sont introduites par les deux injections :

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \text{inl } e : \tau + \sigma} \quad (\text{INL})$$

$$\frac{\Gamma \vdash e : \sigma}{\Gamma \vdash \text{inr } e : \tau + \sigma} \quad (\text{INR})$$

et éliminées par **case** :

$$\frac{\begin{array}{c} \Gamma \vdash e : \tau + \sigma \\ \Gamma, x : \tau \vdash e_1 : v \\ \Gamma, y : \sigma \vdash e_2 : v \end{array}}{\Gamma \vdash \text{case } e \{x.e_1 \mid y.e_2\} : v} \quad (\text{CASE})$$

Les deux branches doivent produire le même type v .

La liste vide est polymorphe :

$$\overline{\Gamma \vdash () : \text{List } \tau} \quad (\text{NIL})$$

La construction d'une liste impose que sa tête et sa queue aient des types compatibles :

$$\frac{\Gamma \vdash x : \tau \quad \Gamma \vdash x' : \text{List } \tau}{\Gamma \vdash x :: x' : \text{List } \tau} \quad (\text{LCONS})$$

La structure correspondant au contexte courant est donnée par :

$$\overline{\Gamma \vdash \text{current-ctx} : \llbracket \Gamma \rrbracket} \quad (\text{CURRENTCTX})$$

Les structures de contextes admettent le sous-typage suivant :

$$\frac{\Delta' \supseteq \Delta}{\llbracket \Delta' \rrbracket <: \llbracket \Delta \rrbracket} \quad (\text{SUBPAIR})$$

Cela signifie que toute structure contenant au moins les variables libres d'une expression est suffisante pour l'évaluer, et donc qu'on peut toujours évaluer une expression dans un contexte plus riche que celui de ses variables libres.

On a donc le sous-typage suivant pour les types de code :

$$\frac{\Gamma \vdash e : \sigma \quad \sigma <: \sigma'}{\Gamma \vdash e : \sigma'} \quad (\text{SUB})$$

c'est-à-dire, par exemple, que si

$$\Delta' = (x : \text{Int}, y : \text{Bool})$$

et

$$\Delta = (x : \text{Int}),$$

alors

$$\llbracket \Delta' \rrbracket <: \llbracket \Delta \rrbracket.$$

Ainsi, si

$$\Gamma \vdash s : \llbracket \Delta' \rrbracket,$$

la règle SUB permet de conclure :

$$\Gamma \vdash s : \llbracket \Delta \rrbracket.$$

La citation transforme une expression typée en code portant son contexte et son type. Γ est le contexte synthétisé pour le terme cité, recensant exactement ses symboles libres, indépendamment de tout ce qui l'entoure :

$$\frac{\Gamma \vdash^{n+1} e : \tau}{\Gamma \vdash^n (\text{quote } e) : \text{Code } \Gamma \tau} \quad (\text{QUOTE})$$

La quasiquotation permet de conserver un contexte externe Δ tout en produisant du code nécessitant Γ . Δ est le contexte ambiant, celui du point du programme où la citation est écrite :

$$\frac{\Delta; \Gamma \vdash^{n+1} e : \tau}{\Delta \vdash^n (\text{quasiquote } e) : \text{Code } \Gamma \tau} \quad (\text{QUASIQUOTE})$$

L'unquote effectue le passage inverse, en consultant précisément ce Δ plutôt que Γ : un symbole ne peut être unquoté que s'il est réellement lié dans le contexte externe, celui où la citation englobante a été écrite.

$$\frac{\Delta \vdash^n e : \text{Code } \Gamma \tau}{\Delta; \Gamma \vdash^{n+1} (\text{unquote } e) : \tau} \quad (\text{UNQUOTE})$$

L'évaluation d'un code ne requiert pas une structure exactement égale à son contexte, mais seulement une structure le couvrant : toute structure plus riche, contenant au moins les symboles attendus, convient par sous-typage de largeur.

$$\frac{\Gamma \vdash a : \text{Code } \Delta \ \tau \quad \Gamma \vdash b : \llbracket \Delta \rrbracket}{\Gamma \vdash (\text{eval-in } a \ b) : \tau} \quad (\text{EVALIN})$$

Cette règle est précisément celle qui empêche l'évaluation d'un code dans une structure dont le contexte ne satisfait pas les variables libres attendues.

L'instanciation permet de passer d'un type polymorphe à une instance :

$$\frac{\Gamma \vdash e : \sigma \quad \sigma \sqsubseteq \sigma'}{\Gamma \vdash e : \sigma'} \quad (\text{INST})$$

La généralisation est autorisée lorsque la variable de type n'apparaît pas librement dans le contexte :

$$\frac{\Gamma \vdash e : \sigma \quad \alpha \notin \text{Free}(\Gamma)}{\Gamma \vdash e : \forall \alpha. \sigma} \quad (\text{GEN})$$

Enfin, **let** introduit la variable typée dans le contexte de son corps :

$$\frac{\Gamma \vdash e_1 : \sigma \quad \Gamma, x : \sigma \vdash e_2 : \tau}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau} \quad (\text{LET})$$

Les règles QUOTE, QUASIQUOTE, UNQUOTE et EVALIN sont les seules extensions nécessaires au traitement typé du code cité ou *homoiconique* ; les autres règles sont celles du λ -calcul typé standard.